

Sirius-Skool — Product Requirements Document

Version: 1.0
Status: Draft
Date: July 3, 2026

Table of Contents

- 1. [Executive Summary](#)
 - 2. [Problem Statement](#)
 - 3. [Scope](#)
 - 4. [User Types & Role Hierarchy](#)
 - 5. [Multi-Tenant Architecture Overview](#)
 - 6. [Functional Modules](#)
 - 6.1 Authentication
 - 6.2 Dashboard
 - 6.3 Student Management & Admission
 - 6.4 Attendance Management
 - 6.5 Result Management
 - 6.6 Notice Board
 - 6.7 Notification System
 - 6.8 Reports
 - 6.9 Settings
 - 7. [Cross-Cutting Modules](#)
 - 7.1 User & Permission Management
 - 7.2 Module Management
 - 7.3 Academic Year Management
 - 8. [Permission Matrix](#)
 - 9. [Core Business Rules](#)
 - 10. [Non-Functional Requirements](#)
 - 11. [Future Scope](#)
 - 12. [Milestones](#)
-

1. Executive Summary

Product Vision

Sirius-Skool is a multi-tenant SaaS School Management System that provides small-to-medium schools with a modular, permission-driven platform to manage their daily operations — from admissions and attendance to results, notices, and reports.

Target Market

- Private K-12 schools
- Tutoring centers and coaching institutes
- Small school chains (2–10 branches)

Business Goals

- Enable rapid tenant onboarding (self-service or assisted)
- Provide modular feature access so schools only pay for/use what they need
- Maintain strict data isolation between tenants
- Keep the permission model simple yet powerful — one `Manager` role with granular module + action assignment

Key Differentiators

- **Modular permission system:** No rigid roles. School Admin creates Managers and assigns specific module+action permissions.
- **Multi-tenant by design:** Every record is scoped to a tenant from day one.
- **Action-level granularity:** View, Create, Edit, Delete, Export — per module, per manager.

2. Problem Statement

Schools currently face one of two extremes:

Problem	Description
Manual operations	Excel sheets, paper registers, manual report generation — slow and error-prone
Over-engineered ERPs	Complex systems designed for large institutions with 20+ roles, unmanageable for small schools

Neither extreme fits a school that wants:

- A simple admin + manager structure
- Modular features (not a bloated all-in-one)
- Affordable SaaS pricing
- Data privacy (their data should not mix with other schools)

How Sirius-Skool Solves This

Need	Solution
Simple roles	School Admin + Manager (permission-based)
Modularity	School Admin enables only needed modules
Multi-tenant	Isolated data per school
Affordable	SaaS model, no on-premise infrastructure
Easy reporting	ID cards, TC, attendance/result reports in one click

3. Scope

In-Scope (MVP)

#	Module	Priority
1	Authentication	P0
2	Dashboard	P0
3	Student Management & Admission	P0
4	Attendance Management	P0
5	Result Management	P1
6	Notice Board	P1
7	Notification System (SMS/Email)	P1
8	Reports	P1
9	Settings	P0

Cross-Cutting (In-Scope)

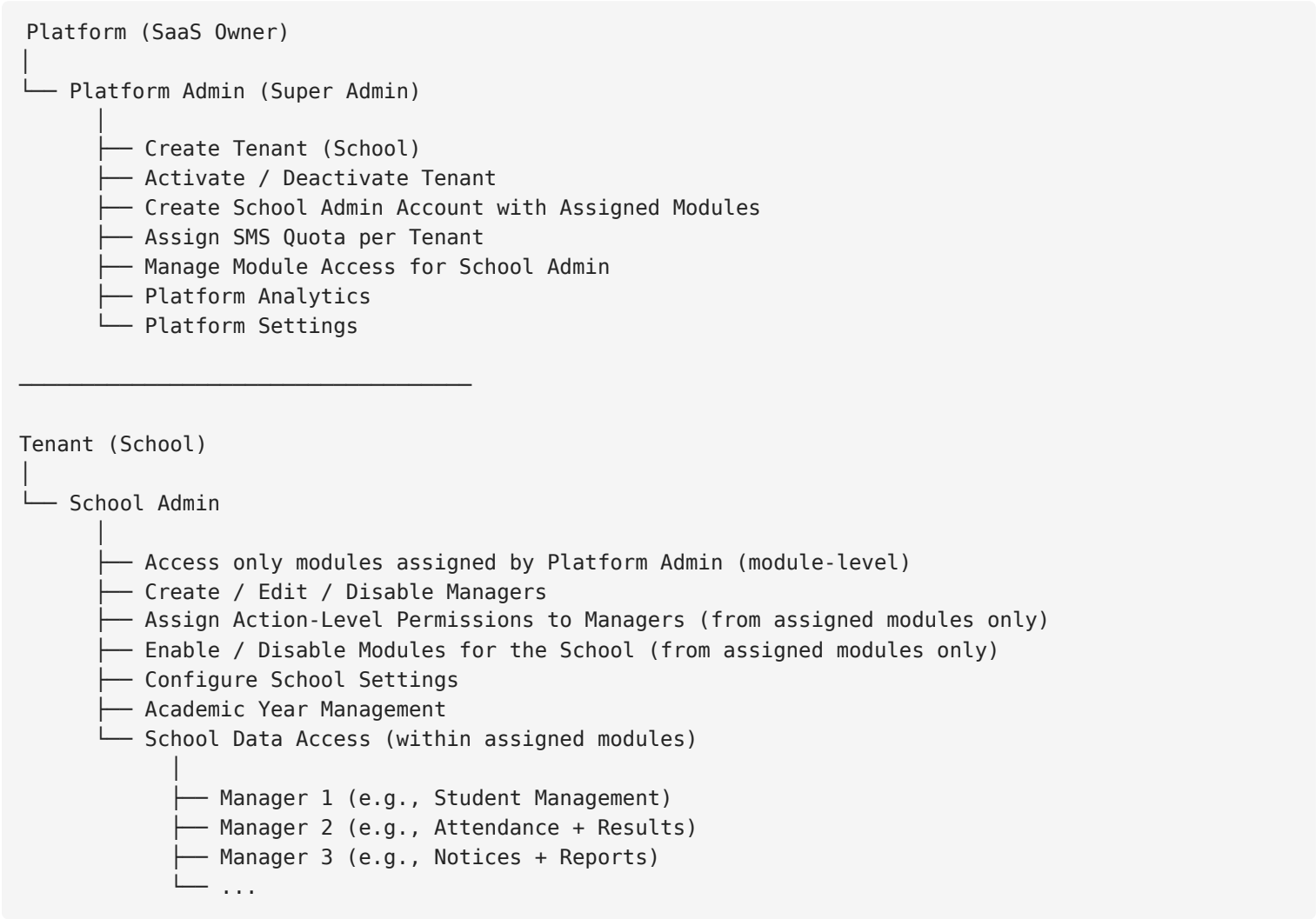
- User & Permission Management
- Module Management (School Admin enables/disables modules)
- Academic Year Management
- Multi-Tenant Data Isolation
- Audit Logging

Out-of-Scope (Post-MVP)

- Fee management / payment gateway
- Parent/student portal
- Library management
- Transport management
- Hostel management

- Mobile apps (iOS/Android)
- Online exams / proctoring
- Payroll / HR management
- Integration with LMS (Google Classroom, Moodle)

4. User Types & Role Hierarchy



Role Definitions

Role	Scope	Creator	Access Model
Platform Admin (Super Admin)	All tenants	System seeded	Full platform control: create tenants, create School Admin accounts, assign modules at module level , set SMS quotas, platform config
School Admin	Single tenant	Platform Admin	Module-level — gets full CRUD + Export on each assigned module. Can then assign action-level permissions to Managers within those modules
Manager	Single tenant	School Admin	Action-level — only assigned View / Create / Edit / Delete / Export on specific modules (from the pool available to School Admin)

Key Rules

- There is exactly **one School Admin per tenant**.
- Platform Admin creates the School Admin account and assigns modules at the **module level** (not action-level). If a module is assigned, School Admin gets all actions on it.
- School Admin assigns permissions to Managers at the **action level** (e.g., View + Create only, no Delete).
- School Admin cannot access or assign any module not granted by Platform Admin.
- School Admin can create any number of Managers and assign them permissions from the available module pool.
- **Platform Admin credentials are stored in a separate platform_admins table** (not the users table). Platform Admin logs in via admin.sirius-skool.com ; school users log in via their tenant subdomain. This separation keeps the platform and tenant authentication systems independent.

5. Multi-Tenant Architecture Overview

Tenant Isolation Model

Recommended: Shared database with tenant_id column on every table, enforced by middleware.

Consideration	Shared DB (tenant_id)	DB-per-Tenant
Cost	Low (one DB server)	High (separate DB per tenant)
Maintenance	Easy (migrations once)	Complex (migrations per DB)
Isolation	Logical (app-enforced)	Physical (full isolation)
Compliance	Adequate for most schools	Required for enterprise/regulated
Backup	Single backup	Per-tenant backup

Recommendation for MVP: Shared DB with tenant_id scoping. Migrate to DB-per-tenant later if enterprise compliance requirements emerge.

Data Ownership Rules

- Every school user record (School Admin, Manager) belongs to exactly one tenant. Platform Admin is stored separately in the platform_admins table and has no tenant_id .
- Every student record belongs to exactly one tenant.
- Every attendance, result, notice, and report belongs to exactly one tenant.
- Cross-tenant data access is prohibited at the application layer.
- The tenant_id is injected into every query via middleware (never trusts user input for tenant scoping).

Tenant Identity (Subdomain Strategy)

Each tenant is identified by a unique subdomain:

- **Format:** `schoolname.sirius-skool.com`
- **Platform Admin URL:** `admin.sirius-skool.com`
- **Auth flow:** The subdomain identifies the tenant at login. Login page detects the subdomain and scopes authentication to that tenant.
- **Wildcard SSL certificate** is required for `*.sirius-skool.com`
- **Future:** Custom domains (`schoolname.com`) can be offered as a paid upgrade

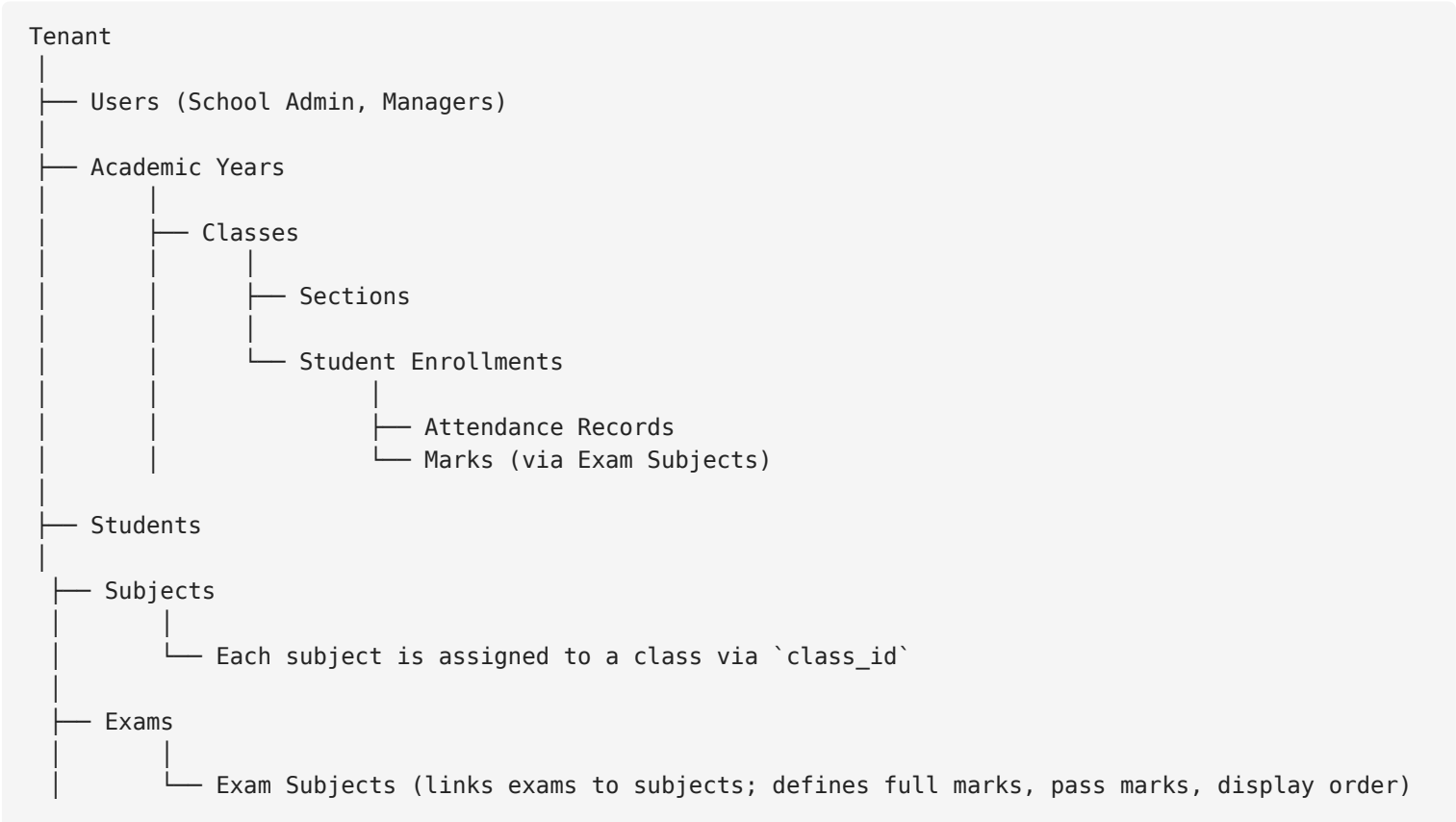
Tenant Lifecycle

Create → Activate → (Active) → Deactivate → (Inactive) → Delete (after X days)

- **Create:** Platform Admin provisions a new school with a unique subdomain, selects which modules the School Admin can access, sets the initial SMS quota, configures the starting registration sequence (if migrating from legacy data), and creates the School Admin account.
- **Activate:** School Admin receives credentials via email, logs in, configures school settings, and begins operations within the assigned modules.
- **Deactivate:** Tenant is suspended (data preserved, login blocked, all sessions invalidated).
- **Delete:** Tenant data is purged after a grace period (soft-delete first, hard-delete after X days).

Core Entity Relationships

The following diagram shows how the core entities relate within a single tenant:



—	Grade Scales
—	Applications (temporary admission table)
—	Notices
—	Settings

Key relationships:

- A **Tenant** has many **Users**, **Students**, **Academic Years**, **Notices**, **Settings**, **Subjects**, **Exams**, and **Grade Scales**.
- An **Academic Year** has many **Classes** (which have **Sections**) and **Student Enrollments**.
- A **Student** has many **Enrollments** across years, but only one active enrollment at a time.
- An **Enrollment** links a **Student** to a **Class** + **Section** within an **Academic Year**, and holds **Attendance** records.
- **Result** data follows a separate chain: **Subjects** are assigned to **Classes** via `subjects.class_id`; **Exams** link to **Subjects** via **Exam Subjects** (which define full marks, pass marks, and display order); and **Marks** are recorded per student enrollment for each exam subject.
- **Grade Scales** define how numeric percentages map to letter grades and GPA points.
- **Applications** are pre-admission records that either convert into a **Student** (on acceptance) or expire.

6. Functional Modules

6.1 Authentication

Purpose: Secure access to the system with role-based routing.

Features:

- Login with email/username + password
- Logout (invalidate session)
- Forgot Password (email-based reset link)
- Change Password (authenticated user)
- JWT-based authentication with access + refresh tokens
- Session management (single session per user optional)
- Role-based redirect after login (School Admin → Admin Dashboard, Manager → Manager Dashboard)
- Tenant-scoped login — user enters subdomain or tenant is identified at login

Business Rules:

- Platform Admin logs in via a separate URL (e.g., `admin.sirius-skool.com`) and authenticates against the `platform_admins` table.

- School Admin and Managers log in via tenant URL (e.g., `schoolname.sirius-skool.com`) and authenticate against the `users` table.
 - Platform Admin credentials are managed independently from school user credentials — password resets, session timeouts, and 2FA (future) are configured separately for each domain.
 - A user cannot be logged into multiple tenants simultaneously.
 - Inactive/deactivated users cannot log in.
-

6.2 Dashboard

Purpose: Provide role-specific at-a-glance overview of key metrics.

Features:

School Admin Dashboard:

- Total student count (current academic year)
- Today's attendance percentage
- Recent notices (last 5)
- Upcoming exams/events
- Manager activity summary
- Remaining SMS quota (e.g., "420 / 500 SMS remaining")
- Quick actions (Add Student, Take Attendance, Create Notice)

Manager Dashboard (permissions-dependent):

- Widgets are shown only for assigned modules
- If assigned Student Management → show student count
- If assigned Attendance → show today's attendance %, pending attendance
- If assigned Results → show recent result entries
- Quick actions relevant to assigned modules

Business Rules:

- Dashboard data is scoped to the current academic year
 - Managers only see data for modules they have View permission on
 - Metrics are displayed in real-time (no caching for MVP)
-

6.3 Student Management & Admission

Purpose: Complete student life cycle management within a tenant.

Features:

Student CRUD:

- Add new student (with photo, guardian details, address, etc.)
- Edit student details
- View student profile (full history)
- Delete student (soft delete with restore option)
- Search by name, roll number, class, registration number
- Filter by class, section, academic year, student status, enrollment status

Student & Enrollment Status:

Status Domain	Values	Description
Student Status	ACTIVE / GRADUATED / TRANSFERRED / DROPPED	Tracks the student's overall standing in the school. Set when the student is admitted (ACTIVE) and updated by end-of-year processing or transfer operations.
Enrollment Status	ACTIVE / COMPLETED / PROMOTED / REPEATED	Tracks the state of a specific enrollment within an academic year. An ACTIVE enrollment becomes PROMOTED or REPEATED at year-end, or COMPLETED on graduation/transfer/dropout.

A student can have multiple enrollments across years, each with its own enrollment status, but only one student status.

Admission:

Admission Workflow:

Student fills admission form → Temporary DB (30 days) → Manager reviews → Accept → Main DB
→ Reject → Deleted from temp DB

- **Step 1 — Online Form:** Student or parent fills an admission form with all required fields (personal info, guardian details, address, documents). This is stored in a **temporary admission table** and assigned a unique **Application ID** (e.g., APP-000001).
- **Step 2 — Temp Storage:** Records in the temporary table auto-expire after 30 days (cron job or TTL index).
- **Step 3 — Review Page:** Manager with admission permission sees a paginated list of pending applications. Can search by Application ID or filter by date/class.
- **Step 4 — Accept:** Moves the student record from the temporary table to the main `students` table, assigns a permanent registration number (YY + sequence , e.g., 26000001), enrolls in the selected class + section, assigns roll number.
- **Step 5 — Reject:** Deletes the record from the temporary table (no trace retained).
- The **Application ID** (APP-000001) is separate from the **permanent registration number** (26000001) — they belong to different domains and share no relationship. The Application ID tracks the application; the registration number is the student's permanent identifier, assigned only on acceptance.

End-of-Year Processing: At the end of each academic year, every student must be assigned one of the following outcomes:

Outcome	Effect on Enrollment	Student Status
Promote	Creates new enrollment in the next class for the new academic year. Roll numbers auto-assigned (adjustable). Bulk/group promotion supported.	ACTIVE
Repeat	Creates new enrollment in the same class for the new academic year. Keeps the existing registration number.	ACTIVE
Graduate	No new enrollment. Student has completed the final year offered by the school.	GRADUATED
Transfer	No new enrollment. Generates a Transfer Certificate (TC). Reason and date recorded.	TRANSFERRED
Dropout	No new enrollment. Records the dropout reason and date.	DROPPED

Bulk Operations:

- Import students from Excel (with template download)
- Export student list to Excel/PDF

Registration Number — Format & Generation:

Attribute	Detail
Format	YY + Sequence (e.g., 26000001)
YY	Last two digits of the admission calendar year (e.g., 26 for 2026, 27 for 2027)
Sequence	Continuous, never-resetting integer per tenant, padded to minimum width
Examples	26000001 , 26000568 , 27001569 , 341000000 (padded width auto-expands)

Generation Mechanism:

- A `current_student_sequence` column on the `tenants` table stores the counter per tenant.
- On admission: read counter → generate `YY + padded(sequence)` → increment counter → commit (all in one atomic transaction).
- The sequence starts at 1 for new tenants. For schools migrating from legacy data, the starting value is configurable at tenant setup.

Registration year vs Academic Year: The registration number uses the **calendar year** of admission (e.g., a student admitted in January 2027 gets `27xxxxx` in the sequence), while the student's enrollment belongs to an **Academic Year** (e.g., `2026-2027`). These are independent concepts — a single Academic Year can contain students admitted across two different calendar years (e.g., a student admitted in June 2026 and another admitted in January 2027 can both be enrolled in Academic Year `2026-2027`).

Roll Number:

Attribute	Detail
Purpose	Class + section daily identifier
Scope	Unique per (class + section + academic year)
Reset	Re-assigned each academic year on promotion
Storage	<code>student_enrollments.roll_number</code> — not in the registration number
Auto-assign	Sequential within section (1–35 for section A, 1–32 for section B)
Override	School Admin can manually adjust individual roll numbers

Section:

- A section is a subgroup within a class (e.g., Class 7 → Sections A, B, C).
- Sections are created by School Admin in Settings.
- A student is enrolled in one class + one section per academic year.
- A student can change section in a new academic year or on promotion.

Data Model (conceptual):

```
classes (id, name, tenant_id)
sections (id, class_id, name, tenant_id)

student_enrollments (
  student_id, class_id, section_id,
  academic_year_id, roll_number,
  tenant_id
)
```

Business Rules:

- Registration number is auto-generated, unique per tenant (enforced by `UNIQUE(tenant_id, registration_number)`), and never changes.
- The class/section is NOT encoded in the registration number — students can be promoted without their permanent ID becoming inaccurate.
- Roll number is distinct from registration number and exists only on the enrollment record.
- A student can have multiple enrollments across academic years, but only one active enrollment at a time.
- Deleting a student requires confirmation and is soft-delete (data preserved for reports).
- Promoting a student creates a new enrollment in the next class for the new academic year; the previous year's enrollment becomes historical.

6.4 Attendance Management

Purpose: Record and track daily student attendance.

Features:

Attendance UI Flow:

1. Manager/Admin selects **Class** → **Section** → **Date**
2. System loads all enrolled students for that class + section, displaying **Name** and **Roll Number**
3. All students are **checked as Present by default**
4. Manager unchecks any student who is **Absent**
5. After marking, Manager clicks "**Save Attendance**"
6. A separate "**Send SMS**" button appears — clicking it sends an absence notification SMS to the guardians of all students marked absent

Other Features:

- Edit attendance within a configurable window (e.g., same day only)
- View attendance history — per class, per student, per date range
- Monthly attendance report (tabular + summary)
- Per-student attendance report (percentage calculation)

Business Rules:

- Attendance is scoped to academic year + class + section
 - All students default to Present — only absentees are explicitly marked
 - Only today's and future dates can have attendance taken (no back-dating beyond 7 days, configurable)
 - Edited attendance is logged (who changed what, when)
 - Once a month is closed (configured by School Admin), attendance becomes read-only
 - "Send SMS" sends only to guardians of absent students (uses system's centralized SMS gateway)
 - Each SMS sent consumes from the tenant's SMS quota (see Notification System)
-

6.5 Result Management

Purpose: Create exams, record marks, and publish results.

Features:

- Create exam types (Midterm, Final, Quiz, etc.)
- Configure subjects per class — subjects are managed centrally and assigned to classes, allowing different classes to have different subject combinations
- Configure full marks, pass marks, and subject weight per exam per class per subject (via exam_subjects)
- Configure grade scales (letter grade + GPA mapping) for automatic grade calculation
- Enter marks per student per subject (numeric or grade-based)
- Auto-calculate total, percentage, rank, letter grade, and GPA
- Publish/unpublish results
- View result sheet (tabular — student-wise or class-wise)
- Generate rank list

- Automatic SMS/Email notification to parents when results are published

Business Rules:

- Results are scoped to academic year + exam + class
- Marks can be entered only for the current academic year
- Once marks are submitted and published, editing requires "unpublish" first
- Published results are visible in reports and student history
- Marks entry can be done by multiple managers (e.g., subject-wise) if permissions allow
- The **Publish** action is exclusive to the School Admin role. Managers can be assigned View, Enter Marks, and Edit Marks, but **not** Publish.

Data Model (conceptual):

The Result module follows a hierarchical workflow:

```
subjects → exam_subjects → marks
                        |
                        └─ grade_scales (lookup)
```

```
subjects (id, name, code, class_id, tenant_id)
  -- Assigns a subject to a class via class_id; a class can have multiple subjects

exams (id, name, type, academic_year_id, class_id, tenant_id)

exam_subjects (id, exam_id, subject_id, full_marks, pass_marks, display_order, tenant_id)
  -- Configures full marks, pass marks, and display order for each subject within an exam

marks (id, exam_subject_id, student_enrollment_id, obtained_marks, is_absent, is_published, tenant_id)
  -- Records the actual score a student received for a specific exam subject

grade_scales (id, tenant_id, name, min_marks, max_marks, grade_letter, grade_point, display_order)
  -- Defines the mapping from numeric marks to letter grades and GPA
```

Key data relationships:

- A **Subject** is assigned to a **Class** via `subjects.class_id` (each subject belongs to one class).
- An **Exam** has multiple **Exam Subjects** (`exam_subjects` references an exam + subject), each with its own full marks, pass marks, and display order — this allows the same exam type (e.g., Midterm) to have different configurations per class.
- A **Mark** is recorded per `student_enrollment` for each `exam_subject` — this eliminates duplication of full/pass marks across student rows.
- **Grade Scales** are a tenant-level lookup: the system calculates the percentage from `obtained_marks` / `full_marks` , looks up the matching grade scale row, and assigns the letter grade and GPA.

6.6 Notice Board

Purpose: Publish and manage notices for staff and students.

Features:

- Create notice (title, body, attachments)
- Publish notice immediately
- Schedule notice for future date (auto-publish)
- Archive notice (hide from active view, keep for records)
- Delete notice (soft delete)
- Filter by date range, status (published/scheduled/archived)
- Notices are visible on the dashboard

Business Rules:

- Notices are tenant-scoped
 - Scheduled notices auto-publish at the configured date/time
 - Published notices cannot be edited (must archive and recreate)
 - Archived notices are retained for audit/report purposes
-

6.7 Notification System

Purpose: Send automated and manual notifications via SMS and Email. The platform provides centralized notification infrastructure — tenants do not configure their own gateways.

Architecture: Centralized Gateway

- The platform (Sirius-Skool) negotiates a bulk rate with SMS (e.g., Twilio) and Email (e.g., SendGrid) providers.
- All tenant notifications are sent through the platform's centralized gateway.
- Usage is tracked per tenant for billing/monitoring.
- This provides a seamless experience — School Admins never configure SMTP or SMS API keys.

SMS Quota System:

- Each tenant has a **SMS quota** (credits) assigned by Platform Admin.
- Platform Admin sets and adjusts the quota from their dashboard (e.g., 500 SMS/month).
- Every SMS sent by the tenant consumes one credit from their quota.
- When quota is exhausted, SMS sending is blocked (Email still works if configured).
- School Admin can view their remaining SMS balance on the dashboard.
- A warning notification is sent to School Admin when balance is low (e.g., < 10% remaining).

Features:

- **SMS:**
 - Attendance notification (absent student)

- Result notification (marks published)
- Custom SMS (ad-hoc, sent by School Admin or authorized Manager)
- **Email:**
 - Attendance notification
 - Result notification
 - Forgot Password / Reset Password (platform-level)
 - Custom email (ad-hoc)
- Notification logs (view sent/failed status per notification with tenant context)
- Retry failed notifications (configurable retry policy per tenant)

Notification Templates:

- Default templates are provided by the platform for each notification type.
- School Admin can customize templates (edit the message body while preserving variables).
- Template variables: `{student_name}` , `{class}` , `{section}` , `{date}` , `{school_name}` , `{percentage}` (results), `{subject}` (results)
- Customizations are per-tenant and do not affect other tenants.

Business Rules:

- Notification failures must not roll back the triggering action (e.g., attendance mark or result publish succeeds even if SMS fails)
- Platform Admin manages the central gateway credentials (not exposed to tenants)
- Platform Admin sets and adjusts SMS quota per tenant from the platform dashboard
- Each SMS sent consumes one credit from the tenant's quota; sending is blocked when quota is 0
- Notification logs are retained per tenant for audit
- Platform Admin can view notification logs across all tenants for support
- Template changes apply to the next notification, not retroactively

6.8 Reports

Purpose: Generate and export documents for students.

Features:

Student Reports:

- ID Card (with photo, name, class, roll number, school details)
- Admission Form (filled admission details)
- Student Profile (complete history)

Certificates:

- Transfer Certificate (TC) — generated on student transfer

- Testimonial / Character Certificate

Academic Reports:

- Attendance Report (per student or per class)
- Result Report (per exam or consolidated)

Export:

- PDF (all reports)
- Excel (attendance/result data)

Business Rules:

- All reports include school branding (logo, name, address from Settings)
 - Reports are generated for the current academic year by default, with option to select previous years
 - Data is read-only at report generation time (reports never modify data)
 - Reports include only data belonging to the current tenant
 - ID card and certificate templates are consistent across the tenant (same layout, auto-filled with school details)
-

6.9 Settings

Purpose: Tenant-level configuration managed by School Admin.

Features:

School Information:

- School name, logo, address, phone, email
- Website URL
- School code/registration number

Academic:

- Academic Year management (create, set active, close)
- Class/Section setup (create classes, sections within classes)
- Subject setup (per class)

Notifications:

- Notification template customization (edit SMS and Email message bodies per notification type)
- Enable/disable notification types (e.g., disable SMS attendance alerts, keep email)
- Sender name/ID override (customize the "from" name shown to recipients)

Security:

- Password policy (minimum length, complexity requirements)
- Session timeout duration

General:

- Language preference (for future i18n)
- Timezone, date format, number format

Registration:

- Set starting registration sequence value (for schools migrating from legacy systems)

Business Rules:

- Settings are per-tenant, never shared
- Only School Admin can modify settings
- Notification template changes apply to the next notification, not retroactively
- Academic Year can only be closed if all attendance and results for that year are finalized
- Gateway credentials are managed by Platform Admin (centralized), not exposed in tenant Settings

7. Cross-Cutting Modules

These are not separate menu items but affect every module and feature in the system.

7.1 User & Permission Management

Purpose: School Admin creates and manages Manager accounts with action-level permissions (limited to modules assigned by Platform Admin at module-level).

Access Model (Two-Tier):

Level	Assigner → Assignee	Granularity
Tier 1	Platform Admin → School Admin	Module-level — if assigned, School Admin gets full access (View, Create, Edit, Delete, Export). No action-level restriction
Tier 2	School Admin → Manager	Action-level — School Admin picks which specific actions the Manager gets (e.g., View + Create only)

Features:

- Create Manager (name, email, password)
- Edit Manager details
- Activate / Deactivate Manager
- Assign **action-level** permissions to Manager (View, Create, Edit, Delete, Export per module)
- Only modules assigned by Platform Admin appear in the list for School Admin to delegate

- View list of all Managers with their permission summary

Permission Model (Tier 2 — School Admin → Manager):

Module: Student Management

└─ View

└─ Create

└─ Edit

└─ Delete

└─ Export

Module: Attendance

└─ View

└─ Take (Create)

└─ Edit

Module: Results

└─ View

└─ Enter Marks (Create)

└─ Edit Marks

└─ Publish

Module: Notice Board

└─ View

└─ Create

└─ Edit

└─ Delete

└─ Schedule

Module: Reports

└─ View

└─ Export

Module: Settings

└─ (School Admin only – not assignable)

Module: User Management

└─ (School Admin only – not assignable)

Business Rules:

- Platform Admin assigns modules at **module-level** — School Admin gets all actions on an assigned module
 - School Admin can only delegate modules that Platform Admin assigned — they cannot delegate what they don't have
 - School Admin assigns permissions to Managers at **action-level** — selecting which specific operations each Manager can perform
 - A Manager with no permissions cannot see any module or data
 - Permissions are additive — granting View + Create means the Manager can both see and create records
 - Deactivating a Manager immediately revokes access (active sessions are invalidated)
-

7.2 Module Management

Purpose: Control which modules are actively used by the school. Module access operates on two tiers:

Tier	Role	Action	Meaning
Tier 1 — Availability	Platform Admin	Assigns modules to the tenant	Determines which modules the school can possibly use . Modules not assigned are invisible to the tenant.
Tier 2 — Activation	School Admin	Enables/disables assigned modules	Determines which available modules are currently active for daily operations. Disabled modules are hidden from the sidebar and inaccessible to all users.

Features:

- School Admin sees all modules assigned by Platform Admin with an individual toggle on/off
- When a module is disabled, it is hidden from the sidebar and inaccessible to all users (including School Admin)
- Disabling a module does not delete associated data
- Modules not assigned by Platform Admin are not visible in this list

Business Rules:

- Authentication and Settings modules cannot be disabled
 - School Admin cannot enable a module that Platform Admin did not assign
 - Disabling a module hides it for the entire tenant
 - Re-enabling a module restores access to existing data
-

7.3 Academic Year Management

Purpose: Every transaction is scoped to an academic year. Schools manage their own academic years.

Features:

- Create Academic Year (name, start date, end date)
- Set one Academic Year as active
- Close Academic Year (make read-only)
- Re-open closed year (if needed)

Business Rules:

- Exactly one Academic Year can be active per tenant at any time
 - All data entry (attendance, results, admissions) happens in the active academic year
 - Previous academic years become read-only after closure — no edits allowed, only viewing reports
 - Students are promoted by creating a new enrollment in the next academic year
 - Academic years cannot overlap in date ranges within a tenant
-

8. Permission Matrix

Two-Tier Model:

- Tier 1 (Platform Admin → School Admin): **Module-level** — if a module is assigned, School Admin gets *all* actions on it. No action-level restriction.
- Tier 2 (School Admin → Manager): **Action-level** — School Admin selects specific actions for each Manager.

Module	Action	Platform Admin	School Admin	Manager
Authentication	Login	✓	✓	✓
	Logout	✓	✓	✓
	Change Password	✓	✓	✓
Dashboard	View	✓	✓	✓ (permissions-dependent)
Student Mgmt	All actions	✗	✓ if module assigned*	As assigned by School Admin
Attendance	All actions	✗	✓ if module assigned*	As assigned by School Admin
Results	All actions	✗	✓ if module assigned*	As assigned by School Admin
Notice Board	All actions	✗	✓ if module assigned*	As assigned by School Admin
Notifications	All actions	✗	✓ if module assigned*	As assigned by School Admin
Reports	All actions	✗	✓ if module assigned*	As assigned by School Admin
Settings	All actions	✗	✓ if module assigned*	✗
User Mgmt	All	✗	✓ (always)	✗
Module Mgmt	All	✗	✓ (always)	✗
Academic Year	All	✗	✓ (always)	✗
Platform	All	✓	✗	✗

- ✓ = Always granted
- ✗ = Never granted
- ✓ if module assigned* = School Admin gets this module (with all its actions) only if Platform Admin assigned it at module-level. If assigned, full CRUD + Export included.
- As assigned by School Admin = Manager gets only the specific actions (View, Create, Edit, Delete, Export) the School Admin chooses to delegate within the available module pool.

9. Core Business Rules

Tenant Rules

1. Every school user, student, and record belongs to exactly one tenant. Platform Admin is an exception — it belongs to the platform and authenticates via the separate `platform_admins` table.

2. Cross-tenant data access is prohibited at every layer (API, DB, UI).
3. Tenant deactivation immediately blocks all user access for that tenant.
4. Platform Admin cannot directly modify tenant operational data (students, attendance, results) — only support/view.

Academic Rules

5. Only one Academic Year can be active per tenant at any time.
6. Students are promoted by creating a new enrollment in the next Academic Year.
7. Previous Academic Years become read-only after closure.
8. All attendance, result, and admission operations happen in the active Academic Year.

Permission Rules (Two-Tier Model)

9. **Tier 1 — Platform Admin → School Admin:** Platform Admin assigns modules at **module-level**. If a module is assigned, School Admin gets all actions (View, Create, Edit, Delete, Export) on that module. No action-level restriction.
10. **Tier 2 — School Admin → Manager:** School Admin assigns permissions at **action-level** within the modules they received from Platform Admin. They can give a Manager View-only, Create-only, or any combination.
11. School Admin can only delegate modules that Platform Admin granted — they cannot delegate what they don't have.
12. A Manager with no assigned permissions cannot see any module or data.
13. Permissions are additive — granting View + Create means the Manager can both see and create records.
14. Deactivating a Manager immediately revokes access across all sessions.

SMS Quota Rules

15. Each tenant has an SMS quota assigned by Platform Admin.
16. Every SMS sent consumes one credit from the tenant's quota.
17. SMS sending is blocked when the quota reaches zero (email still works).
18. Platform Admin can adjust the quota for any tenant from the platform dashboard at any time.

Registration Number Rules

19. Registration number format is `YY + continuous_sequence` (e.g., `26000001`). No class, section, or other changeable attribute is embedded.
20. The sequence is stored per tenant in `tenants.current_student_sequence` and increments atomically on each admission.
21. Registration number is unique per tenant (`UNIQUE(tenant_id, registration_number)`) and never changes for the student's entire tenure.
22. Starting sequence value is configurable at tenant creation for legacy migrations.

Data Rules

- 23. Deleting a record performs a soft delete (data preserved for reports/audit).
- 24. Student, attendance, and result data cannot be permanently deleted once created (only soft-deleted).
- 25. Audit logs are immutable — once written, they cannot be edited or deleted.

Notification Rules

- 26. Notification failures must not roll back the source transaction (attendance, result publish, etc.).
- 27. All notifications are sent through the platform's centralized gateway (no per-tenant gateway configuration).
- 28. Notification logs are retained for troubleshooting.

Reporting Rules

- 29. Reports are generated within the selected Academic Year.
- 30. Reports include only data belonging to the current tenant.
- 31. All reports include school branding from Settings.

10. Non-Functional Requirements

Performance

Why flexible targets? These are initial targets for MVP and will be refined through real-world usage. Actual performance depends on deployment infrastructure (server spec, DB size, network). The architecture is designed to scale horizontally, so these targets can be met by adding resources rather than rewriting code.

Metric	Target (MVP)	Notes
Page load time	< 2–3 seconds (P95)	Target for initial deployment. Can be improved with caching (Redis) and CDN as traffic grows
API response time	< 500ms–1s (P95)	Most CRUD endpoints should respond faster; report generation may take longer
Concurrent users per tenant	50–100+	Depends on school size. Horizontal scaling can increase this linearly
Total tenants (MVP)	100+	Shared DB with tenant_id scales well to hundreds of tenants. Monitor query performance and add read replicas if needed

Security

Requirement	Detail
Authentication	JWT (access + refresh tokens)

Requirement	Detail
Password storage	bcrypt or Argon2
Data encryption	TLS 1.3 in transit, AES-256 at rest
API security	Rate limiting, CORS, CSRF protection
Input validation	All inputs sanitized and validated server-side
Audit logging	All create/update/delete operations logged with user ID, tenant ID, timestamp, IP

Scalability

- Horizontal scaling: application layer can scale independently
- Database: read replicas for reporting queries (future)
- Caching: Redis for session store and frequently accessed data (classes, settings)
- **File storage:** Cloudinary for all file uploads — student photos, ID card templates, notice attachments, report exports
 - Benefits: CDN delivery, automatic image optimization, transformation (resize/crop for ID cards), secure upload with signatures

Availability

Metric	Target
Uptime SLA	99.5% (MVP), 99.9% (post-MVP)
Maintenance window	Off-peak hours with notice
Backup	Daily automated backup, 30-day retention

Data Isolation

- All queries are scoped by `tenant_id` via application middleware
- No raw SQL queries bypass tenant scoping
- Export/import operations are also tenant-scoped

Internationalization (Future)

- Database schema should support multi-language labels (consider using locale tables from day one)
- Date, time, and number formats should be configurable per tenant

11. Future Scope

These features are intentionally excluded from MVP to keep scope focused and deliverable. They are noted here to inform architectural decisions.

Module	Priority	Notes
Fee Management	High	Fee structure, invoices, payment gateway (Stripe/SSLCommerz), due reminders
Parent/Student Portal	High	Separate login for parents/students to view attendance, results, notices, pay fees
Library Management	Medium	Book catalog, issue/return, fines, barcode scanning
Transport Management	Medium	Routes, GPS tracking, stops, fee calculation, vehicle maintenance
Hostel Management	Medium	Room allocation, attendance, fee management, warden dashboard
Payroll / HR	Low	Staff salary, leave management, attendance for staff
Online Exams	Low	Timed online tests, auto-grading, proctoring
Mobile Apps	Low	iOS/Android apps for parents, teachers, students
LMS Integration	Low	Google Classroom, Moodle sync
Multi-Language	Medium	i18n support for UI and reports

Architectural note for future scope: The permission system (module + action) should be designed to easily accommodate new modules without code changes. A new module like "Fee Management" should be auto-discoverable and assignable via the same permission UI.

12. Milestones

Phase 1 — Foundation (MVP)

Milestone	Modules	Duration (est.)
M1.1	Tenant onboarding, Auth, Settings, Academic Year	3–4 weeks
M1.2	User & Permission Management, Module Management	2 weeks
M1.3	Student Management & Admission	3–4 weeks
M1.4	Attendance Management	2 weeks
M1.5	Result Management	2–3 weeks
M1.6	Notice Board, Notifications, Reports	3–4 weeks
M1.7	Dashboard, Audit Logs, Polish, Deployment	2 weeks

Total MVP: ~16–20 weeks (4–5 months)

Phase 2 — Enhancement

Feature	Goal
Fee Management	Revenue operations
Parent/Student Portal	Reduce parent workload on school staff

Feature	Goal
Multi-Language Support	Expand to non-English markets

Phase 3 — Scale

Feature	Goal
Mobile Apps	Increase accessibility
Online Exams	Expand use case
LMS Integration	Compete with larger ERPs
Analytics & BI	Data-driven insights for schools
Subscription & Billing	Automate tenant payments

End of PRD